

# MoMo SMS Transactions API

Building & Securing a REST API — Project Report

African Leadership University

**Enterprise Web Development**  
**Cohort 1\_Team5**

# 1. Introduction to API Security

Modern applications rarely operate in isolation. Mobile apps, web frontends, and partner integrations all need programmatic access to the same underlying data, and APIs are the contract that mediates this access. Because that contract is exposed over the public internet, API security is not optional — it is the property that decides whether the data behind the API stays in the hands of its rightful users.

API security covers several distinct concerns. Authentication answers the question of who is making the request. Authorization decides what that identity is allowed to do once authenticated. Transport security protects credentials and data in flight from eavesdropping or tampering. Input validation prevents malformed or hostile requests from corrupting state or triggering unsafe behavior on the server. Rate limiting and abuse prevention protect the API from being overwhelmed by either malicious traffic or buggy clients. A production-grade API addresses all five; a learning project, by necessity, addresses a subset.

This project implements one slice of that stack — authentication via HTTP Basic Auth — applied to a REST API serving mobile-money SMS records parsed from the MTN MoMo dataset. The choice of Basic Auth is deliberate: it is the simplest credential scheme in the HTTP specification, which makes it ideal for demonstrating the mechanics of authentication without introducing the additional complexity of token issuance, signature verification, or third-party identity providers. The cost of that simplicity, however, is real, and the limitations section of this report examines those costs in detail and recommends what a production system would use instead.

The rest of this report documents the implemented endpoints, presents test evidence for each, analyzes the data-structure choices that shape the API's lookup performance, and reflects honestly on the security gaps that a follow-on iteration would need to close.

## 2. System Overview

The system ingests a backup of MoMo SMS notifications, normalizes the unstructured message bodies into typed transaction records, and exposes those records via a REST API. The pipeline runs in three stages:

- **Parse:** an XML parser reads the SMS backup file, classifies each message into one of five transaction types (RECEIVE, PAYMENT, TRANSFER, DEPOSIT, AIRTIME), and extracts amount, sender, receiver, balance, and timestamp via type-specific regular expressions. Unclassifiable messages are routed to a dead-letter list rather than dropped silently.
- **Serve:** the API loads the parsed records into memory on startup, indexes them by transaction ID in a dictionary, and exposes five CRUD endpoints over HTTP. The server is implemented in plain Python using `http.server`, no web framework, to demonstrate explicit control over request parsing, routing, authentication, and response formatting.
- **Protect:** every request is checked against a Basic Authentication header before any handler is invoked. Failed checks return 401 Unauthorized with a WWW-Authenticate response header, conforming to RFC 7617.

The XML input contains 1,693 SMS records. Of these, the parser successfully classified and extracted structured data from the great majority; the exact counts for parsed records and dead-letter entries are summarized below.

```
hassanah@Hassanats-MacBook-Pro Momo-sms-analytics % python3 etl/parse_xml.py data/raw/modified_sms_v2.xml | head -3
Parsed 1681 transactions, 10 dead-letter entries
By type: {'RECEIVE': 63, 'PAYMENT': 700, 'DEPOSIT': 248, 'TRANSFER': 585, 'AIRTIME': 15, 'MERCHANT_PAYMENT': 36, 'WITHDRAWAL': 3, 'DATA_BUNDLE': 21, 'FAILED': 3, 'REVERSAL': 1, 'TRANSFER_OUT': 6}
Dead-letter reasons: {'ignored_otp_notification': 8, 'unclassified': 2}
hassanah@Hassanats-MacBook-Pro Momo-sms-analytics %
```

## 3. API Endpoints

Five endpoints implement the full CRUD lifecycle on transaction records. All require Basic Authentication. Full request/response examples appear in docs/api\_docs.md; this section summarizes the contract.

| Method | Path               | Purpose                                  | Success Code |
|--------|--------------------|------------------------------------------|--------------|
| GET    | /transactions      | List all transactions                    | 200 OK       |
| GET    | /transactions/{id} | Retrieve one transaction by ID           | 200 OK       |
| POST   | /transactions      | Create a new transaction                 | 201 Created  |
| PUT    | /transactions/{id} | Update fields on an existing transaction | 200 OK       |
| DELETE | /transactions/{id} | Remove a transaction                     | 200 OK       |

### 3.1 Authentication

Credentials are passed in the Authorization header as Basic base64(username:password). The server decodes the header, splits on the first colon, and compares against the configured username and password using hmac.compare\_digest to avoid leaking information through timing differences. A missing or invalid header produces a 401 response with WWW-Authenticate: Basic realm="MoMo API".

### 3.2 Error Handling

The server returns appropriate HTTP status codes for predictable failure modes:

| Status                 | When it happens                                            |
|------------------------|------------------------------------------------------------|
| 400 Bad Request        | Malformed JSON body or missing required fields on POST/PUT |
| 401 Unauthorized       | Missing, malformed, or invalid credentials                 |
| 404 Not Found          | Unknown transaction ID or unrecognized path                |
| 405 Method Not Allowed | HTTP method not supported on the given path                |

Error responses share a uniform JSON shape: `{ "error": "description" }`. This consistency lets clients handle failures with a single parser regardless of the failure type.

## 4. Testing & Validation

Each endpoint was tested with curl against the running server. The screenshots in this section capture both the issued command and the server's response, including the HTTP status line and headers (curl -i flag), so the result is verifiable rather than asserted.

#### 4.1 Successful GET with authentication

Valid credentials, list endpoint returns 200 OK.

```
curl -i -u admin:password123 http://localhost:8000/transactions
```

```

● hassanah@Hassanats-MacBook-Pro Momo-sms-analytics % curl -i -u admin:password123 http://localhost:8000/transactions | head -30
% Total    % Received % Xferd  Average Speed   Time    Time     Time    Current
           Dload  Upload   Total   Spent    Left     Speed                               0HTTTP/1.0 200 OK

Server: BaseHTTP/0.6 Python/3.13.7
Date: Fri, 29 May 2026 12:53:22 GMT
Content-Type: application/json; charset=utf-8
Content-Length: 833774

{
  {
    "id": 1,
    "transaction_type": "RECEIVE",
    "amount": 2000.0,
    "fee": 0.0,
    "sender": "Jane Smith",
    "receiver": null,
    "balance_after": 2000.0,
    "external_ref": "76662021700",
    "timestamp": "2024-05-10 16:30:51",
    "is_transaction": true,
    "raw_body": ". You have received 2000 RWF from Jane Smith (*****013) on your mobile money account at 2024-05-10 16:30:51. Message from sender: . Your new balance:2000 RWF. Financial Transaction Id: 76662021700."
  },
  {
    "id": 2,
    "transaction_type": "PAYMENT",
    "amount": 1000.0,
    "fee": 0.0,
    "sender": null,
    "receiver": "Jane Smith",
    "balance_after": 1000.0,
    "external_ref": "7321484437",
    "timestamp": "2024-05-10 16:31:39",
    5      814k    5 48996    0    0 4054k    0 ---:--:-- ---:--:-- ---:--:-- 3987k

```

## 4.2 Unauthorized request

Invalid credentials are rejected with 401 Unauthorized and a WWW-Authenticate header.

```
curl -i -u admin:wrongpassword http://localhost:8000/transactions
```

```
hassanah@Hassanats-MacBook-Pro Momo-sms-analytics % curl -i -u admin:wrongpassword http://localhost:8000/transactions
HTTP/1.0 401 Unauthorized
Server: BaseHTTP/0.6 Python/3.13.7
Date: Fri, 29 May 2026 12:55:19 GMT
WWW-Authenticate: Basic realm="MoMo API"
Content-Type: application/json; charset=utf-8

{"error": "Unauthorized"}
```

### 4.3 Successful POST

A valid JSON body creates a new transaction; server returns 201 Created with the assigned ID.

```
curl -i -u admin:password123 -X POST http://localhost:8000/transactions \ -H
"Content-Type: application/json" \ -d
'{"transaction_type": "PAYMENT", "amount": 5000, "receiver": "Test User"}'
```

```
hassanah@Hassanats-MacBook-Pro Momo-sms-analytics % curl -i -u admin:password123 -X POST http://localhost:8000/transactions \
-H "Content-Type: application/json" \
-d '{"transaction_type": "PAYMENT", "amount": 5000, "receiver": "Test User"}'

HTTP/1.0 201 Created
Server: BaseHTTP/0.6 Python/3.13.7
Date: Fri, 29 May 2026 12:56:08 GMT
Content-Type: application/json; charset=utf-8
Content-Length: 94

{
  "transaction_type": "PAYMENT",
  "amount": 5000,
  "receiver": "Test User",
  "id": 1684
}
```

## 4.4 Successful PUT

A valid JSON body updates the named fields on an existing transaction; server returns 200 OK with the updated record.

```
curl -i -u admin:password123 -X PUT http://localhost:8000/transactions/1694 \
-H "Content-Type: application/json" \ -d '{"amount": 7500}'
```

```
hassanah@Hassanats-MacBook-Pro Momo-sms-analytics % curl -i -u admin:password123 -X PUT http://localhost:8000/transactions/1 \
-H "Content-Type: application/json" \
-d '{"amount": 7500}'

HTTP/1.0 200 OK
Server: BaseHTTP/0.6 Python/3.13.7
Date: Fri, 29 May 2026 12:56:50 GMT
Content-Type: application/json; charset=utf-8
Content-Length: 465

{
  "id": 1,
  "transaction_type": "RECEIVE",
  "amount": 7500,
  "fee": 0.0,
  "sender": "Jane Smith",
  "receiver": null,
  "balance_after": 2000.0,
  "external_ref": "76662021700",
  "timestamp": "2024-05-10 16:30:51",
  "is_transaction": true,
  "raw_body": "You have received 2000 RWF from Jane Smith (*****013) on your mobile money account at 2024-05-10 16:30:51. Message from sender: . Your new balance:2000 RWF. Financial Transaction Id: 76662021700."
}
```

## 4.5 Successful DELETE

The record is removed and the server returns 200 OK with a confirmation message.

```
curl -i -u admin:password123 -X DELETE
http://localhost:8000/transactions/1694
```

```
hassanah@Hassanats-MacBook-Pro Momo-sms-analytics % curl -i -u admin:password123 -X DELETE http://localhost:8000/transactions/1

HTTP/1.0 200 OK
Server: BaseHTTP/0.6 Python/3.13.7
Date: Fri, 29 May 2026 12:57:26 GMT
Content-Type: application/json; charset=utf-8
Content-Length: 38

{
  "message": "Transaction deleted"
}
```

## 5. Data Structures & Algorithms

Fetching a transaction by ID is the most common read operation in the API. The naive approach is linear search: scan the list of transactions until the matching ID is found. The better approach is to build a dictionary keyed by ID at startup and look up by key. This section measures both approaches empirically to show that the theoretical performance difference holds in practice.

### 5.1 Methodology

Both methods were implemented in `dsa/search_comparison.py`. The benchmark loads the parsed transactions from `data/processed/transactions.json`, takes the first  $N$  records for several values of  $N$ , and times 100 lookups against each method. Each lookup batch is repeated enough times to bring the total measurement above the resolution of `time.perf_counter`, then divided back out to get an average microsecond cost per lookup. Random seed is fixed at 42 so results are reproducible.

### 5.2 Results

| Records (N) | Linear Search ( $\mu$ s) | Dict Lookup ( $\mu$ s) | Speedup  |
|-------------|--------------------------|------------------------|----------|
| 20          | 0.707                    | 0.062                  | 11.4x    |
| 200         | 8.430                    | 0.132                  | 63.9x    |
| 2000        | 93.593                   | 0.064                  | 1451.8x  |
| 20000       | 1334.754                 | 0.068                  | 19738.8x |

### 5.3 Why dictionary lookup beats linear search

Linear search has to compare every element until it finds the target. In the worst case (target absent, or target at the end) it touches all  $N$  elements, so its running time grows in direct proportion to  $N$ . This is the textbook  $O(N)$  complexity, and the benchmark above shows it clearly: doubling  $N$  roughly doubles the lookup time.

Python dictionaries are hash tables. When you write `index[id]`, Python computes a hash of the key, jumps directly to the bucket where that key lives, and returns the value. The work done is independent of how many other entries the dictionary holds, so the time stays roughly flat —  $O(1)$  on average. The benchmark shows dictionary lookup taking around  $0.07 \mu$ s at every dataset size, while linear search rose from  $0.4 \mu$ s to tens of microseconds as  $N$  grew. The speedup factor itself grows linearly with  $N$ : at the full dataset, dictionary lookup is roughly 1,500 $\times$  faster at 2,000 records and nearly 20,000 $\times$  faster at 20,000 records — the gap widens as the dataset grows because linear search scales with  $N$  while dict lookup stays flat.

### 5.4 The tradeoff dictionaries pay

Dictionaries use more memory than a list of the same data because their underlying hash table keeps empty slots by design to keep collisions rare. They also have an upfront construction cost — building the index takes  $O(N)$  — which means the first lookup is not actually faster than a single linear scan. The win only appears when the index is reused across many lookups, which is exactly the API's access pattern: build once on startup, then serve thousands of GET /transactions/{id} requests.

## 5.5 What else could improve search

For lookup by integer ID, a dictionary is already optimal — you cannot beat  $O(1)$  average-case. The question becomes interesting only if the access pattern changes:

- For range queries (e.g., "all transactions between two dates"), a dictionary is the wrong tool because it does not preserve ordering. A sorted list with binary search runs in  $O(\log N)$  for both lookup and range scan, and a B-tree — which is exactly the structure most relational databases use for indexed columns, including the MySQL indexes built in the Week 2 database — extends this to large persistent datasets.
- For prefix searches (e.g., "all senders whose name starts with 'Sam'"), a trie indexed by character beats both a list scan and a dictionary scan because it shares storage among keys with common prefixes.
- For approximate or fuzzy matches (e.g., "all transactions where the receiver name is similar to 'Jane Smyth'"), neither a dictionary nor a sorted list helps directly; the standard answer is an inverted index or a specialized search engine such as Lucene.

The general lesson is that data-structure choice is dictated by the access pattern, not by the dataset. Dictionaries are correct for this API's primary access pattern, but a dashboard that needed to plot transaction volume by date would benefit from a different structure entirely.

## 6. Reflection on Basic Authentication Limitations

Basic Authentication is the right choice for this assignment because it is simple, transparent, and lets the implementation focus on the mechanics of authentication rather than the additional complexity of cryptographic tokens. It is not a defensible choice for a production system. This section explains why, and what a production replacement would look like.

### 6.1 Protocol-level weaknesses

Basic Auth encodes credentials as `base64(username:password)` and sends them in the Authorization header. Base64 is an encoding, not encryption: anyone who captures the header recovers the plaintext credentials with a single decoding step. On any unencrypted connection, this means a passive observer on the network path — including the user's ISP, any compromised intermediate router, or anyone sharing the same Wi-Fi network — can read the credentials directly off the wire.

Even on an HTTPS connection, the credentials are transmitted on every single request rather than once at the start of a session. This multiplies the exposure surface: every request log that records headers, every reverse proxy that buffers requests in memory, and every misconfigured debugging endpoint becomes a potential disclosure point. Token-based schemes avoid this by sending the password exactly once (at login) and using a short-lived token on subsequent requests.

Basic Auth also has no built-in concept of expiry. Credentials remain valid until the user explicitly changes the password, so a leaked password creates an indefinite vulnerability window. Token schemes such as JWT issue tokens with a fixed lifetime — typically minutes for access tokens, days or weeks for refresh tokens — which bounds the damage from a leak.

Finally, Basic Auth is all-or-nothing. A user who can authenticate can hit every endpoint. There is no built-in mechanism for scoped permissions — for example, granting a third-party analytics service read-only access to transactions without also giving it the ability to delete them. OAuth2 was designed specifically to handle this kind of delegated, scoped access.

## 6.2 Implementation-level weaknesses in this project

Beyond Basic Auth's protocol-level limitations, the implementation in this project layers additional weaknesses that a production system would address:

- Credentials are hardcoded in source code (`api/auth.py`). In production, they would be loaded from environment variables or a secrets manager such as AWS Secrets Manager or HashiCorp Vault, and never committed to version control.
- The password is stored as plaintext for direct comparison. Production systems store a salted hash (using `bcrypt`, `scrypt`, or `Argon2`) and compare the hash of the incoming password to the stored hash. This means a leak of the credentials store does not directly expose passwords.
- There is only a single set of credentials shared by all clients. Production systems give each user (or each API consumer) their own credentials, which enables auditing, individual revocation, and per-user rate limiting.

One implementation detail that this project did get right is the use of `hmac.compare_digest` for the credential comparison. Naive string comparison short-circuits on the first mismatched character, which leaks information about the correct value through timing differences observable to an attacker. `hmac.compare_digest` takes constant time regardless of where the strings differ. This is a small detail but it is a real attack vector — see CVE-2018-1000805 and similar — and worth keeping even in a learning project.

## 6.3 Stronger alternatives

Three replacements stand out depending on the deployment context.

JSON Web Tokens (JWT) are signed tokens issued by the server after an initial authentication step. The client stores the token and presents it on every subsequent request in the `Authorization: Bearer` header. The server verifies the signature to confirm the token is genuine, and reads the embedded expiry timestamp to confirm it is still valid. JWT is stateless — the server does not need to store session data — which makes it well-suited to APIs that scale horizontally. Its main drawback is that revocation before natural expiry is hard, since the server has no record of which tokens have been issued.

OAuth2 is a delegation framework rather than a single protocol. It allows a third-party application to obtain a scoped, time-limited token to act on a user's behalf without ever seeing the user's password. OAuth2 is the right choice when external applications need access to the API on behalf of



users — for instance, a budgeting app that needs to read a user's MoMo transaction history. The setup is more complex than Basic Auth or JWT, requiring an authorization server and well-defined scopes, but the security benefits are substantial.

Mutual TLS (mTLS) authenticates clients via cryptographic certificates rather than passwords or tokens. The client presents a certificate during the TLS handshake, the server verifies it against a trusted certificate authority, and only then does the request proceed. mTLS is the standard choice for service-to-service communication inside a closed network — for example, a payment processor talking to a fraud-detection service — where managing certificates is easier than managing user credentials.

All three of these depend on the underlying connection being protected by TLS. None of them — and not even the strongest token-based scheme — provides protection if traffic flows over plaintext HTTP. The first security improvement any deployment of this API would make is to terminate TLS at a reverse proxy and refuse non-TLS connections.

## 7. Conclusion

This project implements a complete CRUD REST API for mobile-money SMS data using only the Python standard library, demonstrating the request parsing, routing, authentication, and response handling that web frameworks normally abstract away. The XML ingestion pipeline classifies messages into five transaction types and routes unparseable records to a dead-letter list. The API exposes five endpoints protected by HTTP Basic Authentication with constant-time credential comparison. A measured benchmark confirms the expected performance gap between linear search and dictionary lookup, growing from 11× at 20 records to nearly 20,000× at 20,000 records.

The security analysis in this report is deliberate: Basic Auth is correct for the assignment's pedagogical goals, but unsuitable for any real deployment. The intended trajectory of a follow-up version is JWT-based authentication over enforced TLS, with credentials migrated to environment variables and password storage migrated to bcrypt hashes. Those changes would close the largest gaps without requiring a fundamental rewrite of the API itself.